

MDL File Format Specification

Format: Blizzard MDL Model Format (Warcraft III / Warcraft III: Reforged) **Encoding:** ASCII / UTF-8 plain text **Dialect:** Warcraft III (engine-faithful) — see [Appendix A](#) for the HiveWorkshop variant **Companion format:** [MDX](#) (binary)

MDL is the **human-readable text representation** of the Warcraft III model format. It carries the same model data as the binary MDX format — geometry, materials, skeletal animation, particle effects, cameras, collision volumes — but as a structured, brace-delimited text file that artists and tools can read and edit directly. The Warcraft III world editor and the game engine convert freely between MDL and MDX.

This document specifies the **Warcraft III dialect** — the text the game's own MDL reader/writer produces and consumes. A second community dialect, used by HiveWorkshop tooling (Retera Model Studio, Magos, MdlVis, mdx-m3-viewer), is described in [Appendix A](#).

Authorship attribution: Fernando A. Sahmkow

Revision History

Date	Version	Description
2026-05-21	1.0	Initial specification of the Warcraft III MDL dialect, derived from the WhiteoutLib MDL writer/parser and validated by a byte-exact MDX→MDL→MDX round-trip over the <code>war3.w3mod</code> game-asset corpus.

Table of Contents

- 1. [Overview](#)
- 2. [Lexical Structure](#)
- 3. [Syntax](#)
- 4. [Animation Tracks](#)
- 5. [Node Properties](#)
- 6. [Top-Level Blocks](#)
- 7. [A Complete Minimal Example](#)

Appendix

- [A — HiveWorkshop Dialect Differences](#)

1. Overview

An MDL file is a sequence of **top-level blocks**, each introduced by a keyword. The file has no magic number and no header — the first non-whitespace token is the `Version` keyword. Blocks may, in principle, appear in any order, but the Warcraft III writer always emits them in the canonical order listed in [Section 6](#), and readers should not depend on any other order.

A block groups a set of **properties**, **flags**, **animation tracks**, and nested **sub-blocks** between braces:

```
Version {
    FormatVersion 800,
}

Model "Footman" {
    BlendTime 150,
    MinimumExtent { -45.0, -45.0, 0.0 },
    MaximumExtent { 45.0, 45.0, 120.0 },
    BoundsRadius 91.0,
}
```

MDL is **whitespace-insensitive** for parsing purposes — indentation and line breaks exist only for readability. The canonical writer indents each nesting level with a single **tab** and places one entry per line.

MDL vs. MDX

MDL is a faithful text mirror of MDX. The two formats hold the same model and the game converts losslessly between them, with these representational notes:

- MDX stores some quantities both as a static scalar **and** as an optional animation track; MDL expresses each property once (either a **static** value or an animated track block).
- Indices that MDX stores as raw **u32** (texture IDs, parent IDs, geoset IDs) appear in MDL either as plain integers or as named sentinels such as **None** and **Multiple**.
- MDL omits properties left at their default value; a reader must apply the documented defaults for any property a block does not mention.

2. Lexical Structure

2.1 Character Set & Whitespace

An MDL file is plain text. Spaces, tabs, carriage returns, and line feeds are **whitespace** and separate tokens; runs of whitespace are equivalent to a single space. Line breaks are not statement terminators.

2.2 Comments

A **//** begins a comment that runs to the end of the line. There is no block-comment syntax.

```
ObjectId 5,           // line comment – ignored by the parser
Parent 0,             // "Bone_Root"
```

2.3 Tokens

Token	Description
Identifier	A letter or <code>_</code> followed by letters, digits, or <code>_</code> (e.g. <code>FilterMode</code> , <code>Bone_Root1</code>). Keywords, property names, and flag names are all identifiers.
Number	An integer or floating-point literal (see 2.4).
String	A double-quoted literal (see 2.5).
<code>{ }</code>	Block / vector delimiters.
<code>,</code>	Entry separator (and trailing separator).
<code>:</code>	Keyframe time/value separator.
<code><=</code>	HD sub-texture slot designator (<code>static TextureID 0 <= 1,</code>).

2.4 Numbers

Numbers are written in standard decimal notation and may be:

- **Integers** — optional sign then digits: `0`, `150`, `-1`, `4294967295`.
- **Floats** — optional sign, digits, `.`, fractional digits, and an optional exponent: `1.0`, `-0.5`, `0.95538312`, `1.5e-08`, `270.0`.
- **Non-finite** — the literals `nan`, `inf`, and `-inf` (case-insensitive). These occur in shipped models (e.g. an unused, uninitialised layer field) and must be accepted by readers.

The canonical writer prints floats using the **shortest decimal string that round-trips back to the exact same 32-bit float**, and always keeps a decimal point on integral values (`50` is written `50.0`). Readers must parse with correct (nearest) rounding so that the value survives the round-trip.

Signed keyframe times. Integer literals used as keyframe times and event times are **signed** — negative frames (e.g. `-3600`) occur in game assets as animation lead-in. Readers must parse them as signed 32-bit integers.

2.5 Strings

A string is any run of characters between double quotes: `"Textures\Armor.blp"`.

- Strings have **no escape sequences** — a backslash is a literal backslash (Windows-style asset paths embed them directly: `"Textures\Diffuse.blp"`).
- A string **may contain embedded line breaks**. PopcornFX emitter strings, in particular, contain literal `CR/LF` characters; the closing quote — not the end of line — terminates the string.

```
Image "ReplaceableTextures\TeamColor\TeamColor00.blp",
```

3. Syntax

Every construct in an MDL file is one of five forms: a **block**, a **property**, a **flag**, a **vector**, or an **animation track**.

3.1 Blocks

A block is a keyword, zero or more **header parameters** (strings or numbers), an opening `{`, a body, and a closing `}`:

```
BlockName [param ...] {
    body
}
```

Header parameters identify or size the block. They are used for:

- a **name**: `Model "Footman", Bone "Bone_Head", Anim "Stand";`
- a **count**: `Sequences 12, Vertices 348, PivotPoints 27;`
- both, or several counts: `Faces 1 1044.`

3.2 Properties

A property is a name, a value, and a trailing comma:

```
BlendTime 150,
FieldOfView 0.7853982,
Image "Textures\Diffuse.blp",
FilterMode Blend,
```

The value is a number, a string, an **identifier** (an enumerated keyword such as `Blend` or `Multiple`), or a **vector** (3.4). A property whose value is a vector is written `Name { ... },.`

3.3 Static Properties

Some properties may be either a fixed value or animated over time. When fixed, they are written with the `static` keyword prefix; when animated, they appear as an **animation track** of the same name:

```
static Alpha 1.0,                                // fixed
```

```
Alpha 3 {                                          // animated – same property, track form
    Linear,
    0: 1.0,
    500: 0.0,
    1000: 1.0,
}
```

A reader encountering `static Name value` stores `value` as the property's base value with no track; encountering `Name count { ... }` stores an animation track.

3.4 Flags & Vectors

A **flag** is a bare identifier with a trailing comma — its mere presence sets a boolean: **NonLooping**,, **Billboarded**,, **TwoSided**,, **DropShadow**,.

A **vector** is a brace-enclosed, comma-separated list of numbers. Vectors are fixed-length by context: 2 for UV coordinates, 3 for positions/colours, 4 for quaternions, 12 for bind-pose matrices:

```
MinimumExtent { -45.0, -45.0, 0.0 },
Color { 1.0, 0.5, 0.25 },
{ 0.0, 0.0, 0.0, 1.0 },           // quaternion (x, y, z, w)
```

3.5 Data Blocks

A data block holds an array of anonymous values — vectors or bare numbers — each followed by a comma. The header parameter is the element count:

```
Vertices 3 {
    { 0.0, 0.0, 0.0 },
    { 1.0, 0.0, 0.0 },
    { 0.0, 1.0, 0.0 },
}
```

```
VertexGroup {
    0,
    0,
    1,
}
```

A trailing comma after the final entry is conventional and expected.

4. Animation Tracks

An animation track stores a property's value as a series of **keyframes**. Any property documented as *animatable* may appear in this form.

4.1 Track Syntax

```
PropertyName keyframeCount {
    InterpolationType,
    [GlobalSeqId index,]
    time: value,
    time: value,
```

```
...
}
```

- **keyframeCount** — the header parameter — is the number of keyframes.
- **InterpolationType** is one of **DontInterp**, **Linear**, **Hermite**, **Bezier** (4.2).
- **GlobalSeqId** is present only when the track is driven by a global sequence ([Section 6.4](#)); it gives the global-sequence index. When absent, the track plays on the normal sequence timeline.
- Each keyframe is **time: value**, — **time** is a signed integer in milliseconds, **value** matches the property's type (scalar, vector, or quaternion).

```
Translation 3 {
  Linear,
  0: { 0.0, 0.0, 0.0 },
  333: { 0.0, 0.0, 40.0 },
  1000: { 0.0, 0.0, 0.0 },
}
```

4.2 Interpolation Types

Keyword	Meaning
DontInterp	Stepped — the value holds until the next keyframe.
Linear	Linear interpolation between keyframes.
Hermite	Hermite spline; each keyframe carries in/out tangents.
Bezier	Bézier spline; each keyframe carries in/out tangents.

4.3 Tangents (Hermite & Bezier)

For **Hermite** and **Bezier** tracks, every keyframe is followed by an indented **InTan** and **OutTan** line, each carrying a value of the same type as the keyframe:

```
Rotation 2 {
  Hermite,
  0: { 0.0, 0.0, 0.0, 1.0 },
    InTan { 0.0, 0.0, 0.0, 1.0 },
    OutTan { 0.0, 0.0, 0.0, 1.0 },
  1000: { 0.0, 0.70710677, 0.0, 0.70710677 },
    InTan { 0.0, 0.70710677, 0.0, 0.70710677 },
    OutTan { 0.0, 0.70710677, 0.0, 0.70710677 },
}
```

4.4 Event Tracks

Event objects (Section 6.19) use a simplified track that stores only frame times — there is no interpolation type and no values:

```
EventTrack 3 {
    800,
    2133,
    4067,
}
```

5. Node Properties

Many objects — Bone, Light, Helper, Attachment, the emitters, EventObject, CollisionShape — share a common **node** substructure that places the object in the scene hierarchy and animates its transform. Wherever a block below is described as *"begins with the node properties,"* it admits the following, in this order:

Property	Form	Description
ObjectId	ObjectId N,	Unique node index. Always present.
Parent	Parent N,	Parent node's ObjectId. Omitted for a root node (a model may have several roots).
Behaviour flags	bare flags	Any of: DontInheritTranslation, DontInheritRotation, DontInheritScaling, Billboarded, BillboardedLockX, BillboardedLockY, BillboardedLockZ, CameraAnchored.
Translation	track (float[3])	Animated local translation.
Rotation	track (float[4])	Animated local rotation, quaternion (x, y, z, w).
Scaling	track (float[3])	Animated local scaling.

The three transform channels are written **only when animated** — a static node omits them entirely (its transform is identity, offset by its pivot point).

```
Bone "Bone_Forearm" {
  ObjectId 4,
  Parent 3,
  Rotation 2 {
    Linear,
    0: { 0.0, 0.0, 0.0, 1.0 },
    1000: { 0.0, 0.0, 0.38268, 0.92388 },
  }
}
```

Every node also has a corresponding entry in the model's **PivotPoints** block, indexed by **ObjectId**.

6. Top-Level Blocks

The Warcraft III writer emits blocks in the order below. All blocks are optional except **Version** and **Model**.

6.1 Version

```
Version {
  FormatVersion 800,
}
```

FormatVersion is the format version: **800** (Classic), or **900/1000/1100/ 1200** (Reforged). It governs which version-gated properties are valid — see the [MDX specification](#).

6.2 Model

```
Model "Arthas" {
  BlendTime 150,
  MinimumExtent { -175.945, -147.584, -73.0451 },
  MaximumExtent { 172.425, 175.079, 553.363 },
  BoundsRadius 0.0,
}
```

Property	Description
(name)	The header parameter is the model name.
BlendTime	Animation cross-fade time in milliseconds; omitted when 0.
MinimumExtent / MaximumExtent / BoundsRadius	Model bounding volume.

`MinimumExtent`, `MaximumExtent`, and `BoundsRadius` together form an **extent** — the same triple recurs in `Sequences` and `Geoset`. The model-level `BoundsRadius` is frequently `0.0` even when the extent box is non-empty.

6.3 Sequences

A `Sequences` block lists the model's named animations. Its header parameter is the sequence count; each child `Anim` block names one sequence.

```
Sequences 2 {
  Anim "Stand" {
    Interval { 0, 3333 },
    MoveSpeed 0.0,
    MinimumExtent { -50.0, -50.0, 0.0 },
    MaximumExtent { 50.0, 50.0, 100.0 },
    BoundsRadius 80.0,
  }
  Anim "Walk" {
    Interval { 3334, 6667 },
    NonLooping,
    MoveSpeed 270.0,
    Rarity 0.0,
    MinimumExtent { -55.0, -55.0, 0.0 },
    MaximumExtent { 55.0, 55.0, 105.0 },
    BoundsRadius 85.0,
  }
}
```

Property	Description
<code>Interval { start, end }</code>	Sequence time range, in milliseconds, on the global timeline.
<code>NonLooping</code>	Flag — sequence plays once instead of looping.
<code>MoveSpeed</code>	Movement speed this animation is authored for; omitted when <code>0</code> .
<code>Rarity</code>	Random-selection weight; omitted when <code>0</code> .
extent	Per-sequence bounding volume.

6.4 GlobalSequences

Global sequences are looping timelines independent of the model's animations; tracks reference them by index via `GlobalSeqId`. Each `Duration` is the loop length in milliseconds.

```
GlobalSequences 2 {
  Duration 1000,
  Duration 2500,
}
```

6.5 Textures

Each **Bitmap** sub-block declares one texture slot.

```
Textures 3 {
  Bitmap {
    Image "Textures\Footman.blp",
  }
  Bitmap {
    Image "",
    ReplaceableId 1,
  }
  Bitmap {
    Image "Textures\Footman_Detail.blp",
    WrapWidth,
    WrapHeight,
  }
}
```

Property	Description
Image	Texture file path. Empty when the slot is a replaceable texture.
ReplaceableId	Replaceable-texture ID (1 = team colour, 2 = team glow, ...); omitted when 0 .
WrapWidth / WrapHeight	Flags — wrap the texture on the U / V axis.

6.6 Materials

A **Materials** block lists **Material** definitions; each material owns one or more **Layer** sub-blocks rendered in order.

```
Materials 1 {
  Material {
    PriorityPlane 0,
    Layer {
      FilterMode None,
      static TextureID 0 <= 0,
    }
    Layer {
      FilterMode Additive,
      Unshaded,
      static TextureID 2 <= 0,
      static Alpha 0.8,
    }
  }
}
```

Material properties

Property	Description
<code>PriorityPlane</code>	Render-order priority; omitted when <code>0</code> .
<code>ConstantColor</code>	Flag.
<code>TwoSided</code>	Flag — render both faces.
<code>Unfogged</code>	Flag — not affected by fog.
<code>SortPrimsNearZ</code> / <code>SortPrimsFarZ</code>	Flags — primitive sort order.
<code>FullResolution</code>	Flag.
<code>Shader</code>	<code>Shader "name"</code> , — material-level HD shader-pipeline name. Present only for version 900 and 1000 (see <i>Shader</i> below).

Layer properties

A `Layer` selects a texture and the way it is blended.

Property	Form	Description
<code>FilterMode</code>	identifier	<code>None</code> , <code>Transparent</code> , <code>Blend</code> , <code>Additive</code> , <code>AddAlpha</code> , <code>Modulate</code> , <code>Modulate2x</code> .
Shading flags	bare flags	<code>Unshaded</code> , <code>SphereEnvMap</code> , <code>TwoSided</code> , <code>Unfogged</code> , <code>NoDepthTest</code> , <code>NoDepthSet</code> , <code>WrapWidth</code> , <code>WrapHeight</code> , <code>Unlit</code> .
<code>Shader</code>	<code>Shader "name"</code> ,	Per-layer HD shader name, written for <code>version ≥ 1100</code> HD layers — see <i>Shader</i> below. Omitted for SD layers.
<code>TextureID</code>	<code>static TextureID id</code> <code><= slot</code> ,	Texture binding — see below. Animatable.
<code>TVertexAnimId</code>	<code>TVertexAnimId N</code> ,	Index into <code>TextureAnims</code> ; omitted when none.
<code>CoordId</code>	<code>CoordId N</code> ,	UV-set index in the geoset; omitted when <code>0</code> .
<code>Alpha</code>	<code>static Alpha a</code> ,	Layer opacity. Animatable; omitted when <code>1.0</code> .
<code>EmissiveGain</code>	<code>static EmissiveGain</code> <code>g</code> ,	Reforged PBR emissive gain. Animatable; default 1.0 .
<code>FresnelColor</code>	<code>static FresnelColor</code> <code>{ r,g,b }</code> ,	Reforged PBR fresnel colour. Animatable; default <code>{ 1,1,1 }</code> .
<code>FresnelOpacity</code> / <code>FresnelTeamColor</code>	<code>static ...</code>	Reforged PBR fresnel scalars. Animatable.

Texture binding. In the Warcraft III dialect a texture is bound with the slot-designator form:

```
static TextureID 0 <= 0,
```

The number after **TextureID** is the index into the **Textures** block; the number after **<=** is the layer's texture **slot**. A classic SD layer uses slot **0** only. A Reforged HD layer (**version** $\geq$ **1100**) carries one binding per slot — diffuse **0**, normal **1**, ORM **2**, emissive **3**, team-colour **4**, reflections **5**:

```
Layer {
    FilterMode None,
    Shader "Shader_HD_DefaultUnit",
    static TextureID 0 <= 0,
    static TextureID 1 <= 1,
    static TextureID 2 <= 2,
    static TextureID 3 <= 3,
    static TextureID 4 <= 4,
    static TextureID 5 <= 5,
    static Alpha 1.0,
}
```

An animated (flipbook) texture binding uses the track form, with no slot designator:

```
TextureID 4 {
    DontInterp,
    0: 2,
    200: 3,
    400: 4,
    600: 5,
}
```

Shader. Where the HD shader name is written depends on the model version:

- **version 900 and 1000** — the shader name is written **once at the material level**, as a **Shader "name"**, property directly inside the **Material** block (before its **Layer** sub-blocks). This placement is identical in both the Warcraft III and the HiveWorkshop dialects.
- **version $\geq$ 1100** — the material-level field no longer exists; instead each HD **Layer** carries its own **Shader "name"**, directive. SD layers omit it.

```
Material {                                     // version 900 / 1000
    Shader "Shader_HD_DefaultUnit",
    Layer {
        FilterMode None,
        ...
    }
}
```

Common shader names: "Shader_HD_DefaultUnit", "Shader_HD_Crystal", "Shader_SD_FixedFunction". The HiveWorkshop dialect marks `version ≥ 1100` HD layers differently — see [Appendix A.1](#).

6.7 TextureAnims

UV-animation definitions, referenced by a layer's `TVertexAnimId`. Each `TVertexAnim` holds up to three transform tracks.

```
TextureAnims 1 {
  TVertexAnim {
    Translation 2 {
      Linear,
      0: { 0.0, 0.0, 0.0 },
      1000: { 1.0, 0.0, 0.0 },
    }
  }
}
```

`Translation` and `Scaling` are `float[3]` tracks; `Rotation` is a `float[4]` quaternion track.

6.8 Geoset

A `Geoset` is one renderable mesh. There is one `Geoset` block per mesh — they are **not** wrapped in a counted parent block.

```
Geoset {
  Vertices 4 {
    { -16.0, -16.0, 0.0 },
    { 16.0, -16.0, 0.0 },
    { 16.0, 16.0, 0.0 },
    { -16.0, 16.0, 0.0 },
  }
  Normals 4 {
    { 0.0, 0.0, 1.0 },
    { 0.0, 0.0, 1.0 },
    { 0.0, 0.0, 1.0 },
    { 0.0, 0.0, 1.0 },
  }
  TVertices 4 {
    { 0.0, 0.0 },
    { 1.0, 0.0 },
    { 1.0, 1.0 },
    { 0.0, 1.0 },
  }
  VertexGroup {
    0,
    0,
    0,
    0,
  }
}
```

```

    }
    Faces 1 6 {
        Triangles {
            { 0, 1, 2, 0, 2, 3 },
        }
    }
    Groups 1 1 {
        Matrices { 0 },
    }
    MaterialID 0,
    SelectionGroup 0,
    MinimumExtent { -16.0, -16.0, 0.0 },
    MaximumExtent { 16.0, 16.0, 0.0 },
    BoundsRadius 22.63,
}

```

Element	Description
Vertices <i>N</i> { ... }	<i>N</i> vertex positions (<i>float</i> [3]).
Normals <i>N</i> { ... }	<i>N</i> vertex normals (<i>float</i> [3]).
TVertices <i>N</i> { ... }	A UV set — <i>N</i> texture coordinates (<i>float</i> [2]). The block repeats once per UV set.
VertexGroup { ... }	One matrix-group index per vertex (<i>u8</i> each).
Tangents <i>N</i> { ... }	Reforged — <i>N</i> tangents (<i>float</i> [4], <i>xyz</i> + handedness).
SkinWeights <i>N</i> { ... }	Reforged HD skinning — <i>N</i> rows of 8 integers per vertex (4 bone indices + 4 weights, 0–255). Carried alongside the legacy <i>VertexGroup</i> / <i>Groups</i> data, not instead of it. In the Warcraft III dialect each row is written as bare, comma-separated integers — no per-vertex braces (see Appendix A.5).
Faces <i>G</i> <i>T</i> { Triangles { { ... } } }	<i>G</i> triangle groups, <i>T</i> total indices; the <i>Triangles</i> sub-block lists the index buffer.
Groups <i>G</i> <i>M</i> { Matrices { ... }, ... }	<i>G</i> matrix groups containing <i>M</i> total matrix indices; each <i>Matrices</i> { ... } lists one group's bone-matrix indices.
MaterialID	Index into the <i>Materials</i> block.
SelectionGroup	Selection-group index.
Unselectable	Flag — geoset cannot be selected.
LevelOfDetail	Reforged LOD index.
extent	Geoset bounding volume.
Anim { extent }	Per-sequence geoset bounding volumes — one <i>Anim</i> sub-block per sequence.

6.9 GeosetAnim

Animates a geoset's visibility and vertex colour.

```
GeosetAnim {
    static Alpha 1.0,
    DropShadow,
    GeosetId 0,
    static Color { 1.0, 1.0, 1.0 },
}
```

Property	Description
Alpha	Geoset opacity. Animatable.
DropShadow	Flag.
GeosetId	Index of the affected geoset.
Color	Vertex colour { b, g, r }. Animatable; written only when used.

6.10 Bone

A **Bone** is a skinning node. It begins with the [node properties](#) and adds two links:

```
Bone "Bone_Root" {
    ObjectId 0,
    GeosetId Multiple,
    GeosetAnimId None,
}
Bone "Bone_Spine" {
    ObjectId 1,
    Parent 0,
    GeosetId 0,
    GeosetAnimId 1,
}
```

Property	Description
GeosetId	Index of the geoset this bone belongs to, or Multiple if it spans several.
GeosetAnimId	Index of the associated GeosetAnim , or None .

6.11 Light

Begins with the node properties; declares a light source.

```
Light "LightOmni" {
    ObjectId 12,
    Parent 0,
    Omnidirectional,
```

```
static AttenuationStart 50.0,  
static AttenuationEnd 200.0,  
static Color { 1.0, 0.9, 0.7 },  
static Intensity 1.0,  
static AmbColor { 1.0, 1.0, 1.0 },  
static AmbIntensity 0.0,  
}
```

The light type is a bare flag: **Omnidirectional**, **Directional**, or **Ambient**. **AttenuationStart**, **AttenuationEnd**, **Color**, **Intensity**, **AmbColor**, **AmbIntensity**, and **Visibility** are all animatable. **Color** and **AmbColor** default to { 1, 1, 1 }.

6.12 Helper

A helper is a pure transform node — it begins with, and contains only, the [node properties](#).

```
Helper "Overhead Ref" {  
    ObjectId 8,  
    Parent 0,  
}
```

6.13 Attachment

An attachment point for equipment, effects, or spawned models.

```
Attachment "Origin Ref "  
    ObjectId 40,  
    AttachmentID 0,  
}
```

Property	Description
AttachmentID	Attachment slot index.
Path	Attached model path; omitted when empty.
Visibility	Animatable visibility track.

6.14 PivotPoints

One pivot point — the node's local origin — per node, in **ObjectId** order.

```
PivotPoints 3 {  
    { 0.0, 0.0, 0.0 },  
    { 0.0, 0.0, 60.0 },  
}
```

```
{ 0.0, 0.0, 110.0 },  
}
```

6.15 ParticleEmitter

A classic emitter that spawns model instances.

```
ParticleEmitter "ModelEmitter" {  
    ObjectId 20,  
    Parent 0,  
    EmitterUsesMdl,  
    static EmissionRate 8.0,  
    static Gravity 0.0,  
    static Longitude 90.0,  
    static Latitude 0.0,  
    Path "Objects\Spawnmodel\spawn.mdl",  
    static LifeSpan 1.0,  
    static InitVelocity 0.0,  
}
```

`EmitterUsesMdl` / `EmitterUsesTga` are flags. `EmissionRate`, `Gravity`, `Longitude`, `Latitude`, `LifeSpan`, `InitVelocity`, and `Visibility` are animatable.

6.16 ParticleEmitter2

The quad (billboard) particle emitter.

```
ParticleEmitter2 "Smoke" {  
    ObjectId 21,  
    Parent 0,  
    SortPrimsFarZ,  
    Unshaded,  
    static Speed 100.0,  
    static Variation 0.5,  
    static Latitude 45.0,  
    static Gravity 9.8,  
    LifeSpan 2.0,  
    static EmissionRate 50.0,  
    static Length 16.0,  
    static Width 16.0,  
    Additive,  
    Rows 1,  
    Columns 1,  
    Head,  
    TailLength 0.0,  
    Time 0.5,  
    SegmentColor {  
        Color { 1.0, 1.0, 1.0 },  
        Color { 1.0, 0.5, 0.0 },  
    }
```

```

    Color { 0.0, 0.0, 0.0 },
  }
  Alpha { 255, 255, 0 },
  ParticleScaling { 1.0, 2.0, 0.0 },
  LifeSpanUAnim { 0, 0, 1 },
  DecayUAnim { 0, 0, 1 },
  TailUAnim { 0, 0, 1 },
  TailDecayUAnim { 0, 0, 1 },
  TextureID 0,
}

```

Element	Description
Behaviour flags	<code>SortPrimsFarZ</code> , <code>LineEmitter</code> , <code>Unfogged</code> , <code>ModelSpace</code> , <code>Unshaded</code> , <code>XYQuad</code> .
<code>Speed ... Width</code>	Emission parameters; <code>Speed</code> , <code>Variation</code> , <code>Latitude</code> , <code>Gravity</code> , <code>EmissionRate</code> , <code>Length</code> , <code>Width</code> are animatable.
Filter mode	A bare flag: <code>Blend</code> , <code>Additive</code> , <code>Modulate</code> , <code>Modulate2x</code> , <code>AlphaKey</code> .
<code>Rows / Columns</code>	Texture-atlas grid.
Head/tail	A bare flag: <code>Head</code> , <code>Tail</code> , <code>Both</code> .
<code>SegmentColor</code>	A sub-block of exactly three <code>Color { r, g, b }</code> entries (start / mid / end).
<code>Alpha { a, a, a }</code>	Three 0–255 segment alphas.
<code>ParticleScaling { s, s, s }</code>	Three segment scale factors.
<code>LifeSpanUAnim / DecayUAnim / TailUAnim / TailDecayUAnim</code>	UV-animation intervals { <code>start</code> , <code>end</code> , <code>repeat</code> }.
<code>TextureID</code>	Index into <code>Textures</code> .
<code>Squirt</code> , <code>PriorityPlane</code> , <code>ReplaceableId</code>	Optional integers, written only when non-zero.

6.17 RibbonEmitter

A trailing-ribbon emitter.

```

RibbonEmitter "Trail" {
  ObjectId 22,
  Parent 6,
  static HeightAbove 4.0,
  static HeightBelow 4.0,
  static Alpha 1.0,
  static Color { 1.0, 1.0, 1.0 },
  LifeSpan 0.5,
  TextureSlot 0,
}

```

```
EmissionRate 30,
Rows 1,
Columns 1,
MaterialID 1,
}
```

HeightAbove, **HeightBelow**, **Alpha**, **Color**, **TextureSlot**, and **Visibility** are animatable. **Gravity** is written only when non-zero.

6.18 ParticleEmitterPopcorn

A Reforged PopcornFX emitter (the MDL keyword for the MDX **CORN** chunk).

```
ParticleEmitterPopcorn "PopcornFire" {
  ObjectId 23,
  Parent 0,
  static LifeSpan 1.0,
  static EmissionRate 1.0,
  static Speed 1.0,
  static Color { 1.0, 1.0, 1.0 },
  static Alpha 1.0,
  Path "SharedFX\Hero_Glow\Hero_Glow.pkfx",
  AnimVisibilityGuide "Always=on, Death=off",
}
```

Property	Description
Flags	SortPrimsFarZ , Unshaded , Unfogged , PopcornScaling .
LifeSpan , EmissionRate , Speed , Alpha	Animatable scalars; default 1.0 .
Color	Animatable RGB; default { 1, 1, 1 }.
ReplaceableId	Written only when non-zero.
Path	The .pkfx PopcornFX effect path.
AnimVisibilityGuide	Comma-separated Sequence=on/off rules; may contain embedded line breaks.

6.19 EventObject

Fires sound, footprint, or splat effects at specific frames. Begins with the node properties, then an **EventTrack** ([Section 4.4](#)):

```
EventObject "SNDxFootmanFootstep" {
  ObjectId 25,
  Parent 2,
  EventTrack 4 {
```

```
        800,
        2133,
        4067,
        5400,
    }
}
```

6.20 Camera

```
Camera "Portrait" {
    Position { 0.0, -180.0, 90.0 },
    FieldOfView 0.7853982,
    FarClip 1000.0,
    NearClip 0.1,
    Target {
        Position { 0.0, 0.0, 70.0 },
    }
}
```

Element	Description
Position	Camera eye position. Translation adds an animation track.
FieldOfView	Vertical FOV in radians.
FarClip / NearClip	Clip-plane distances.
Rotation	Optional scalar (f32) roll-angle track.
Target { Position ... }	Look-at target; Translation inside it adds a track.

6.21 CollisionShape

A collision volume. Begins with the node properties, then a shape-type flag and its geometry.

```
CollisionShape "Collision" {
    ObjectId 26,
    Parent 0,
    Sphere,
    Vertices 1 {
        { 0.0, 0.0, 60.0 },
    }
    BoundsRadius 50.0,
}
```

The shape type is a bare flag — Box, Plane, Sphere, or Cylinder. Box and Plane carry two Vertices; Sphere carries one; Cylinder carries two. BoundsRadius is present only for Sphere and Cylinder.

6.22 FaceFX

A Reforged facial-animation binding.

```
FaceFX "Face" {
    Path "war3mapImported\face.facefx",
}
```

6.23 BindPose

Reforged bind-pose matrices — one 3×4 (12-float) affine matrix per node.

```
BindPose {
    Matrices 2 {
        { 1.0, 0.0, 0.0, 0.0, 1.0, 0.0, 0.0, 0.0, 1.0, 0.0, 0.0, 0.0 },
        { 1.0, 0.0, 0.0, 0.0, 1.0, 0.0, 0.0, 0.0, 1.0, 0.0, 0.0, 60.0 },
    }
}
```

7. A Complete Minimal Example

A valid, complete MDL file for a single textured quad:

```
Version {
    FormatVersion 800,
}
Model "Quad" {
    MinimumExtent { -16.0, -16.0, 0.0 },
    MaximumExtent { 16.0, 16.0, 0.0 },
    BoundsRadius 22.63,
}
Sequences 1 {
    Anim "Stand" {
        Interval { 0, 1000 },
        MinimumExtent { -16.0, -16.0, 0.0 },
        MaximumExtent { 16.0, 16.0, 0.0 },
        BoundsRadius 22.63,
    }
}
Textures 1 {
    Bitmap {
        Image "Textures\Quad.blp",
    }
}
Materials 1 {
    Material {
        Layer {
            FilterMode None,
            static TextureID 0 <= 0,
        }
    }
}
```

```

    }
  }
}
Geoset {
  Vertices 4 {
    { -16.0, -16.0, 0.0 },
    { 16.0, -16.0, 0.0 },
    { 16.0, 16.0, 0.0 },
    { -16.0, 16.0, 0.0 },
  }
  Normals 4 {
    { 0.0, 0.0, 1.0 },
    { 0.0, 0.0, 1.0 },
    { 0.0, 0.0, 1.0 },
    { 0.0, 0.0, 1.0 },
  }
  TVertices 4 {
    { 0.0, 1.0 },
    { 1.0, 1.0 },
    { 1.0, 0.0 },
    { 0.0, 0.0 },
  }
  VertexGroup {
    0,
    0,
    0,
    0,
  }
  Faces 1 6 {
    Triangles {
      { 0, 1, 2, 0, 2, 3 },
    }
  }
  Groups 1 1 {
    Matrices { 0 },
  }
  MaterialID 0,
  SelectionGroup 0,
  MinimumExtent { -16.0, -16.0, 0.0 },
  MaximumExtent { 16.0, 16.0, 0.0 },
  BoundsRadius 22.63,
}
Bone "Bone_Root" {
  ObjectID 0,
  GeosetID 0,
  GeosetAnimID None,
}
PivotPoints 1 {
  { 0.0, 0.0, 0.0 },
}

```

Appendix A — HiveWorkshop Dialect Differences

The community tool ecosystem (Retera Model Studio, Magos' War3 Model Editor, MdIVis, mdx-m3-viewer) reads and writes a slightly different MDL dialect. It is identical to the Warcraft III dialect described above **except** in the areas below. A reader that wants to accept both dialects should treat the constructs in this appendix as alternative spellings of the same data.

A.1 HD Layer Shader (version ≥ 1100)

For **version** 900 and 1000 the two dialects record the HD shader **identically** — a single material-level **Shader "name"**, property inside the **Material** block (see [Section 6.6](#)). This is *not* a dialect difference.

The dialects diverge only for **version** ≥ **1100**, where the material-level field no longer exists and the shader is recorded **per layer**:

- the **Warcraft III dialect** writes a per-layer **Shader "name"**, directive (**"Shader_HD_DefaultUnit"**, ...);
- the **HiveWorkshop dialect** writes a per-layer numeric **ShaderTypeId N**, instead (**0** = SD, **1** = HD) — and writes no per-layer **Shader** name.

```
Layer {                                     // version >= 1100, HiveWorkshop
    FilterMode None,
    ShaderTypeId 1,
    ...
}
```

A.2 HD Texture Slots — Named Properties

The Warcraft III dialect binds every HD texture slot with the **static TextureID id <= slot**, designator form. The HiveWorkshop dialect uses a **distinct property name per slot** and never writes the **<=** designator:

Slot	Warcraft III dialect	HiveWorkshop dialect
0 Diffuse	<code>static TextureID 0 <= 0,</code>	<code>static TextureID 0,</code>
1 Normal	<code>static TextureID 1 <= 1,</code>	<code>static NormalTextureID 1,</code>
2 ORM	<code>static TextureID 2 <= 2,</code>	<code>static ORMTTextureID 2,</code>
3 Emissive	<code>static TextureID 3 <= 3,</code>	<code>static EmissiveTextureID 3,</code>
4 Team Colour	<code>static TextureID 4 <= 4,</code>	<code>static TeamColorTextureID 4,</code>
5 Reflections	<code>static TextureID 5 <= 5,</code>	<code>static ReflectionsTextureID 5,</code>

Each named-slot property is animatable in the usual track form (**NormalTextureID 4 { ... }**).

```
Layer {
    FilterMode None,
    ShaderTypeId 1,
    static TextureID 0,
```

```

static NormalTextureID 1,
static ORMTexureID 2,
static EmissiveTextureID 3,
static TeamColorTextureID 4,
static ReflectionsTextureID 5,
static Alpha 1.0,
}

```

A.3 Material Flag Names

- The HiveWorkshop dialect writes the far-Z primitive sort flag as `SortPrimitives`, rather than `SortPrimsFarZ`.
- It **omits** the engine-only material flag `SortPrimsNearZ`, which its tools do not recognise. The `Unfogged` material flag is written by **both** dialects.

A.4 Layer Flag Names

The HiveWorkshop dialect omits the engine-only layer shading flags `WrapWidth`, `WrapHeight`, and `Unlit`, which its tools do not recognise. All other layer flags (`Unshaded`, `SphereEnvMap`, `TwoSided`, `Unfogged`, `NoDepthTest`, `NoDepthSet`) are written identically in both dialects.

A.5 SkinWeights Row Form

Both dialects write a Reforged HD geoset's `SkinWeights` block with the same header (`SkinWeights N { ... }`) and the same payload — 8 integers per vertex (4 bone indices + 4 weights). They differ only in how each vertex's row is delimited:

- the **Warcraft III dialect** writes each row as **bare, comma-separated integers** with no enclosing braces — the engine's MDL reader requires this; a leading `{` makes its parse loop stop early and reject the model;
- the **HiveWorkshop dialect** wraps each vertex row in `{ ... }`, the form its community tooling expects.

```

SkinWeights 3 {                                // Warcraft III dialect
    8, 7, 0, 0, 191, 64, 0, 0,
    8, 0, 0, 0, 255, 0, 0, 0,
    76, 4, 0, 0, 252, 3, 0, 0,
}

```

```

SkinWeights 3 {                                // HiveWorkshop dialect
    { 8, 7, 0, 0, 191, 64, 0, 0 },
    { 8, 0, 0, 0, 255, 0, 0, 0 },
    { 76, 4, 0, 0, 252, 3, 0, 0 },
}

```

A.6 Geoset Selection and LOD Properties

The HiveWorkshop dialect adds two **Geoset**-level properties the Warcraft III dialect never writes:

- **SelectionFlags** *N*, — raw selection flags, written when the geoset's selection state is not simply the **Unselectable** flag. The Warcraft III dialect records only the **Unselectable** flag and has no raw-flags spelling.
- **LevelOfDetailName** *"name"*, — a string label for the LOD. Both dialects write the numeric **LevelOfDetail** *N*, index; only the HiveWorkshop dialect adds the accompanying name.

A.7 Summary

Aspect	Warcraft III dialect	HiveWorkshop dialect
HD layer shader marker (version <i>≥ 1100</i>)	per-layer Shader <i>"name"</i> ,	per-layer ShaderTypeId <i>N</i> ,
HD texture slots	static TextureID <i>id</i> <i><= slot</i> ,	named per-slot properties (NormalTextureID , ...)
HD SkinWeights rows	bare comma-separated integers	each row wrapped in { ... }
Far-Z sort flag	SortPrimsFarZ	SortPrimitives
SortPrimsNearZ material flag	written	omitted
WrapWidth / WrapHeight / Unlit layer flags	written	omitted
Geoset SelectionFlags raw-flags property	omitted	written
Geoset LevelOfDetailName property	omitted	written

The material-level **Shader** *"name"*, of **version** 900/1000 models is **the same in both dialects** and is therefore not listed above. Everything else — block structure, the **static**/track property model, vectors, animation tracks, node properties, number and string lexing, and the full set of top-level blocks — is identical between the two dialects.